Laboratorium 1

Temat: Definiowanie prostych klas, wykorzystanie metod statycznych, praca z debuggerem

Zadanie 1.
Utwórz klasę Program, która zawiera:
- metodę main,
- metodę statyczną sayHello(), wypisującą „Witaj w Javie!”,

Uruchom program i zaprezentuj jego działanie.

Zadanie 2.
Zdefiniuj klasę Student, która zawiera:
- pola: imie, nazwisko, numerIndeksu,
- pole statyczne: liczbaStudentow
- metodę przedstawSie(), wyświetlającą imie, nazwisko oraz numer indeksu.
- metodę statyczną liczbaStudentow(), zwracającą wartość pola liczbaStudentow.
- konstruktor, który inicjalizuje wartości pól imie, nazwisko, numerIndeksu oraz inkrementuje wartość statycznego pola liczbaStudentow

Napisz program tworzący kilka obiektów i dla każdego z nich wywołaj metodę przedstawSie() oraz zaprezentuj działanie metody liczbaStudentow().

Zadanie 3.
Utwórz statyczną klasę MathUtils ze statycznymi metodami:
- silnia(int n) – obliczanie n!,
  - wykorzystaj odpowiednią pętlę
- suma(int a, int b) – zwracającą sumę dwóch liczb,
- srednia(double[] tab) – zwracającą średnią wartości w tablicy.

Zaprezentuj działanie tych metod z poziomu metody main().

Zadanie 4.
Skorzystaj z debuggera w IDE i zaprezentuj jego działanie:
- ustaw breakpoint w metodzie silnia z poprzedniego zadania,
- wykonaj program krok po kroku (Step Into, Step Over),
- obserwuj zmienne lokalne podczas działania pętli,

Zadanie 5.
Zaprojektuj prostą klasę Kalkulator, która udostępnia metody:

- dodaj(double a, double b),
- odejmij(double a, double b),
- pomnoz(double a, double b),
- podziel(double a, double b),

oraz metodę statyczną opis(), która wyświetla nazwę i autora programu.

Przygotuj program, który zaprezentuje działanie ww. metod.

Zadanie 6.
Utwórz klasę KonwerterTemperatur z metodami statycznymi:

- naFahrenheita(double celsjusz),
- naCelsjusza(double fahrenheit).

Przygotuj program testujący metody zgodnie ze wzorcem Arrange-Act-Assert.

Przygotuj program, który wczyta wartość temperatury od użytkownika i przeliczy ją w obie strony, a następnie je wyświetli rezultat w tabeli.

Laboratorium 2

Temat: Definiowanie prostych klas, tworzenie obiektów.

Zadanie 1.
Zdefiniuj klasę, która reprezentuje użytkownika systemu informatycznego.
Polami klasy są:

- login,
- haslo,
- rola (np. administrator, operator, gość)
    - do zdefiniowania tego pola proszę użyć typu enum

Następnie dodaj:

- konstruktor ustawiający wszystkie pola,
- metodę wyswietlDane() — wypisuje login i rolę,
- metodę zmienHaslo(String noweHaslo).

W metodzie main():

- utwórz kilku użytkowników,
- zmień hasło jednego z nich,
- wypisz dane wybranego użytkownika.

Zadanie 2.
Zdefiniuj klasę Serwer, która reprezentuje serwer w centrum danych.

Polami klasy są:

- adresIP,
- nazwa,
- status (np. zatrzymany, wstrzymany, działa, awaria)
    - do zdefiniowania tego pola proszę użyć enum-a
- obciazenieCPU (w %),
- temperatura (w °C).

Dodaj:

- konstruktor inicjalizujący wszystkie pola,
- metodę wyswietlStatus(),
- metodę zwiekszObciazenie(int procent)
    - zwiększa obciążenie CPU i symuluje wzrost temperatury o 0.5°C na każde 10% wzrostu
    - dodatkowo wykorzystując generator pseudolosowy (klasa Random) do symulowania wystąpienia awarii.

W metodzie main():

- utwórz dwa serwery,
- „zwiększ” ich obciążenie,
- wypisz status oraz obciążenie po każdej operacji

Zadanie 3
Zdefiniuj klasę CzujnikTemperatury, która przechowuje dane z czujnika IoT.

Polami klasy są:
- id,
- lokalizacja,
- wartoscOstatniegoOdczytu (temperatura w °C).

Dodaj:
- konstruktor ustawiający wszystkie pola,
- metodę odczyt(), która wypisuje aktualny pomiar
    - wykorzystaj generator pseudolosowy (klasa Random) do symulowania odczytów
- metodę czyPrzegrzany(), która zwraca true, jeśli temperatura > 50°C.

W main():
- utwórz 2 obiekty CzujnikTemperatury
- dla pierwszego obiektu odczytuje wartość pomiaru co 5 sekund (wykorzystaj w tym celu klasę TimeUnit) i zakończ odczyt temperatury po 10 iteracjach.
- dla drugiego obiektu odczytuje wartość temperatury aż do momentu przegrzania.

Zadanie 4.
Zdefiniuj klasę Polaczenie, która przechowuje podstawowe informacje o połączeniu z bazą danych.

Polami klasy są:
- nazwaBazy,
- adresSerwera,
- port,
- uzytkownik,
- polaczenieAktywne

Dodaj kilka konstruktorów:
- bezparametrowy
    - ustawia domyślnie:
        - localhost, port = 3306,
        - uzytkownik = root,
        - czyAktywnePolacznie = true
- z jednym parametrem nazwaBazy,
- z pełnym zestawem parametrów.

Dodaj metody:
- polacz()
    - wypisuje komunikat: "Nawiązano połączenie z bazą <nazwaBazy> na serwerze <adresSerwera>:<port>",

        - zmienia wartość pola polaczenieAktywne na true
- rozlacz().
    - zmienia wartość pola polaczenieAktywne na false

W metodzie main():
- utwórz obiekty korzystając z różnych konstruktorów,
- wywołaj polacz() i rozlacz() dla każdego z nich.

Zadanie 5.
Dodaj do klasy z zadania 1 dodaj pole typu AdresIP.

Zdefiniuj nową klasę AdresIP z polami:
- adres,
- maska,

oraz metodą
- toString().

Zaktualizuj metodę wyswietlDane(), aby uwzględniała adres IP.

Laboratorium 3

Temat: Praca przy wykorzystaniu tablic (jednowymiarowe / dwuwymiarowe).

Zadanie 1

Napisz program, który:

- Tworzy tablicę int[] scores o długości 10. Reprezentuje ona wyniki z egzaminu 10 studentów;
- Wypełnia ją losowymi wartościami z zakresu <0–100>.
- Oblicza i wyświetla:
  - średnią otrzymanych punktów (average),
  - wartość maksymalną (max) i minimalną (min),
  - liczbę studentów, którzy zdali (próg zdania ≥ 51 pkt).
- Wyświetla wyniki w czytelnej formie w konsoli.

Zadanie 2

Utwórz tablicę int[][] grades = new int[5][4]; - gdzie wiersze odpowiadają studentom, a kolumny przedmiotom. Program powinien:

- Wypełnić tablicę losowymi wartościami z przedziału <2,5>.
- Obliczyć średnią ocen każdego studenta (studentAverage) oraz średnią ocen każdego przedmiotu (subjectAverage).
- wyświetlić dane w formie tabeli

```
Student | Subject1 | Subject2 | Subject3 | Subject4 | Average
-------------------------------------------------------------
1       |    4     |    3     |    5     |    4     |   4.0
2       |    5     |    5     |    4     |    5     |   4.75
```

- zawierać metodę int bestStudent(int[][] grades), która zwraca indeks najlepszego studenta. Metoda ta zostanie wywołana jako ostatnia.

Zadanie 3

1. Utwórz tablicę double[][] matrix = new double[5][]; - w której każdy wiersz ma inny rozmiar.
2. Wypełnij ją losowymi wartościami z zakresu 0 - 10.
3. Zaimplementuj metody:
   a. void fillMatrix(double[][] matrix) - wypełnia przekazaną macierz losowymi wartościami
   b. void printMatrix(double[][] matrix) - drukuje przekazaną macierz
   c. double findMax(double[][] matrix) - znajduje najwyższą wartość w przekazanej macierzy

Zadanie 4.
Napisz program, który będzie symulował działanie systemu pomiaru temperatury.
System monitoruje 5 czujników przez 8 godzin pracy. Odczyt z czujnika jest dokonywany raz na godzinę.

- wartości odczytów mają być przechowywane w zmiennej sensors
    - double[][] sensors = new double[5][8];
- wypełnij tablicę losowymi wartościami z zakresu 20.0 - 40.0,
- oblicz średnią temperaturę dla każdego czujnika (averageSensorTemperature(int sensorId)),
- znajdź czujnik o najwyższej średniej (findSensorWithHighestAvg),
- znajdź godzinę, w której zanotowano najwyższą temperaturę (findTimeOfMaxTemperature).

Zadanie 5
Zaimplementuj prostą grę konsolową Kółko i krzyżyk, która:

- Utworzy planszę i ją wydrukuje.
- Umożliwi dwóm graczom na zmianę wstawiać X lub O poprzez wprowadzenie informacji do konsoli.
- Po każdym ruchu wyświetli planszę.
- Sprawdzi, czy któryś z graczy wygrał - jeśli tak, wyświetli o tym informację.
- Jeśli wszystkie pola są zajęte i nikt nie wygrał - wyświetli o tym informację.

W tym zadaniu oceniana jest umiejętność poprawnego nazewnictwa metod, klas, zmiennych, poprawna implementacja wyżej opisanej logiki.

Laboratorium 4

Temat: Agregacja obiektów.

Zadanie 1
Utwórz klasy:
1. Server (adres IP, ilość RAM, liczba procesorów)
2. HardDrive (pojemność, typ, prędkość).

Każdy Server ma jeden lub więcej dysków HardDrive.
Zaimplementuj metody
1. addHardDrive(...) - dodaje nowy dysk,
2. showHardwareInfo() - wyświetla dane serwera i wszystkich jego dysków.

W main() utwórz kilka serwerów o różnych konfiguracjach.

Zadanie 2
Utwórz klasy:
1. UserAccount - (login, hasło, status aktywności),
2. AdminPanel - agreguje i zarządza listą użytkowników.

Klasa AdminPanel powinna umożliwiać:
1. dodanie użytkownika (addUser()),
2. zablokowanie użytkownika (blockUser(login)),
3. wyświetlenie listy kont (showUsers()).

W main() utwórz panel administratora i przetestuj działanie metod.

Zadanie 3
Zaprojektuj klasy:
1. BookingSystem,
2. Employee,
3. Device (np. laptop, tablet, projektor)

BookingSystem agreguje obiekty Employee i Device
1. bookDevice(Employee e, Device d) - przypisuje urządzenie do pracownika,
2. releaseDevice(Device d) - zwraca urządzenie do puli dostępnych,
3. showActiveBookings() - wyświetla aktualne rezerwacje.

Zadanie 4
Zaprojektuj klasy:
1. DataCenter - (lokalizacja, liczba serwerów),
2. Server,
3. NetworkSwitch (model, liczba portów).

Każdy DataCenter agreguje wiele serwerów i przełączników.

Dodaj do klasy DataCenter metody:
1. addServer(),
2. addSwitch(),
3. showInfrastructure().

W main() utwórz centrum danych z różnymi elementami infrastruktury.

Zadanie 5
Utwórz klasy:
1. Team - zespół projektowy,
2. Developer - pracownik (imię, specjalizacja, poziom doświadczenia),
3. Project - projekt informatyczny (nazwa, technologia, termin zakończenia).

Każdy Team może realizować wiele projektów i mieć wielu programistów.

Zaimplementuj metody:
1. addDeveloper() - dodaje programistę,
2. addProject() - przypisuje projekt,
3. showTeamInfo() - wyświetla członków zespołu i aktualne projekty.

Zadanie 6
Zaprojektuj klasy:
1. Database - (nazwa, typ np. SQL / NoSQL),
2. Table - (nazwa, liczba kolumn, liczba rekordów).

Klasa Database agreguje obiekty Table.

Dodaj do klasy Database metody:
1. addTable(),
2. showTables(),
3. getTotalRecords() - zwracającą sumę rekordów ze wszystkich tabel.

W main() utwórz bazę z kilkoma tabelami i wyświetl podsumowanie.

Zadanie 7
Zaprojektuj klasy:

1. WebApplication - (nazwa, wersja, status działania),
2. Module - (nazwa, opis, wersja),
3. Developer – (imię, rola, projekt przypisany).

Każda WebApplication składa się z modułów, a każdy moduł ma przypisanego developera.

Zaimplementuj metody:

1. addModule(Module m) - dodaje moduł,
2. assignDeveloperToModule(Module m, Developer d) - przypisuje developera do modułu,
3. showModules() – wypisuje moduły i ich twórców.

Zadanie 8
Utwórz klasy:

1. Firewall,
2. Rule (reguła dostępu: port, adres IP, akcja: allow/deny),
3. Network (adres podsieci, liczba urządzeń).

Firewall ma listę reguł bezpieczeństwa oraz jest powiązany z określoną Network. Każdy obiekt Network jest powiązany z jednym obiektem Firewall. A obiekt Firewall może posiadać wiele ról.

Zaimplementuj:

1. addRule() - dodaje regułę,
2. showRules() - wyświetla wszystkie zasady,
3. filterConnection(ip, port) - sprawdza, czy ruch z danego IP i portu jest dozwolony.

Laboratorium 5

Temat: Budowa hierarchii klas (klasy abstrakcyjne / dziedziczenie)

Zadanie 1
Zaprojektuj zestaw klas modelujących sprzęt komputerowy w firmie IT.

Utwórz klasę bazową Device, posiadającą:
- prywatne pola: brand, model, year,
- konstruktor ustawiający te pola,
- metodę getInfo() wypisującą informacje o urządzeniu.

Utwórz klasy dziedziczące:
- Laptop
- Mobile

Każda z nich dodaje własne pole (np. batteryLife albo screenSize) oraz przesłania metodę getInfo() (użyj super).

W funkcji main:
- Utwórz tablicę typu Device[] i umieść w niej obiekty obu klas.
- Wywołaj getInfo() dla każdego elementu - ma zadziałać polimorfizm.

Zadanie 2
Zaprojektuj system dla firmy zatrudniającej pracowników.

Utwórz abstrakcyjną klasę Employee, zawierającą:
- pola: name, salary,
- konstruktor,
- metodę abstrakcyjną calculateBonus(),
- zwykłą metodę printInfo().

Utwórz klasy:
- Developer– bonus roczny to 20% pensji,
- Manager – bonus roczny to 50% pensji.

W main:
- Utwórz tablicę Employee[],
- wypełnij różnymi typami pracowników,
- wyświetl ich bonusy.

Zadanie 3
Stwórz hierarchię:

Abstract class Content
↑
Article extends Content
↑
TechnicalArticle extends Article

1. Klasa Content:
    - posiada title, author,
    - ma abstrakcyjną metodę render().
2. Article:
    - dodaje pole text,
    - implementuje render() – wypisuje tytuł i treść.
3. TechnicalArticle :
    - dodaje pole technology,
    - przesłania render(), dopisując informację o technologii.

W main:
1. Zdefiniuj tablicę Content[],
2. umieść w tablicy obiekty Article i TechnicalArticle ,
3. wywołaj render().

Zadanie 4
Stwórz interfejs Deployable, zawierający metodę:
- void deploy();

Następnie zdefiniuj klasy - mają implementować Deployable.:
- BackendService,
- FrontendApp

Utwórz klasę DeploymentManager, która ma metodę:
- public static void runDeployment(Deployable deploy)
    - public static void runDeployment(Deployable d)

W main:
- stwórz kilka obiektów różnych typów,
- przekaż je do runDeployment().
    - ma zadziałać jak wzorzec projektowy Strategia, czyli jeden interfejs, a wiele różnych zachowań.

Zadanie 5
Zaprojektuj swoją mini-hierarchię minimum 1 klasa abstrakcyjna + 2 klasy pochodne + 1 interfejs, np.:

- System modułów logowania,
- System płatności,
- Hierarchia pojazdów autonomicznych,
- Biblioteka multimediów.

Wymagania:

- użyj polimorfizmu,
- użyj interfejsu,
- utwórz tablicę danych typu klasy abstrakcyjnej,
- przesłoń co najmniej jedną metodę,
- przygotuj metodę wykorzystującą instanceof,
- dodaj 1 metodę abstrakcyjną.

Laboratorium 6

Temat: Budowa hierarchii klas (dziedziczenie / polimorfizm - tablice polimorficzne / odwołania polimorficzne)

Zadanie 1
Zaprojektuj hierarchię klas modelującą ogólny system pojazdów:

1. Utwórz klasę abstrakcyjną Vehicle, posiadającą:
    - pola: brand, year,
    - metodę abstrakcyjną: double getFuelConsumption(),
    - metodę: void printInfo() wypisującą podstawowe dane pojazdu.
2. Utwórz klasy dziedziczące:
    - Car,
    - Motorcycle,
    - Truck

Każda z nich implementuje własną wersję metody getFuelConsumption().

W main():

- Utwórz tablicę typu Vehicle[] o długości 6.
- Wypełnij ją różnymi pojazdami.
- W pętli wykonaj printInfo() oraz wypisz wynik getFuelConsumption().

Zadanie 2
Rozbuduj system pojazdów z zadania 1:

1. Dodaj interfejs Electric z metodą:
    - int getBatteryLevel().
2. Dodaj klasę ElectricCar, która dziedziczy po Car i implementuje Electric.
3. Dodaj klasę HybridCar, która również dziedziczy po Car, ale posiada:
    - dwa pola: fuelLevel, batteryLevel,
    - przesłoniętą metodę getFuelConsumption() (np. zależną od poziomu baterii).

W main():

- Umieść wszystkie nowe obiekty w tablicy typu Vehicle[].
- W pętli:
    - wywołaj metody wspólne,
    - jeśli pojazd implementuje Electric, wypisz poziom baterii (użyj instanceof).

Zadanie 3

1. Stwórz klasę Workshop, która:
    - posiada metodę statyczną service(Vehicle v).
        i. metoda powinna wykonywać:
            1. dla Car → „Wymiana oleju”
            2. dla Motorcycle → „Smarowanie łańcucha”
            3. dla Truck → „Kontrola hamulców”
            4. dla ElectricCar → „Kontrola baterii”
2. Użyj:
    - polimorfizmu,
    - instanceof tylko tam, gdzie konieczne.
3. Dodaj 6 różnych pojazdów do tablicy Vehicle[] i wykonaj Workshop.service(...) dla każdego.

Zadanie 4

Zaprojektuj ZOO:

1. Utwórz abstrakcyjną klasę Animal z metodami:
    - abstract void makeSound(),
    - void printInfo().
2. Utwórz klasy:
    - Lion,
    - Elephant,
    - Monkey.
3. Każde zwierzę przesłania makeSound().

W main():

1. Umieść zwierzęta w tablicy Animal[].
2. Dla każdego wykonaj makeSound() oraz printInfo().

Zadanie 5

Zaprojektuj mały system płatności:

1. Interfejs PaymentMethod z metodą:
    - boolean process(double amount).
2. Utwórz klasy implementujące interfejs PaymentMethod:
    - CreditCard,
    - PayPal,
    - Blik,
    - CryptoTransfer.

W main():

- przygotuj tablicę PaymentMethod[],
- spróbuj przetworzyć różne płatności.

Zadanie 6
Zaprojektuj mały system graficzny:

1. Zdefiniuj interfejs Renderable z metodą:
    - void render().
2. Dodaj klasy implementujące ten interfejs:
    - Sprite,
    - TextLabel,
    - RectangleShape,
    - ImageTexture.
3. Dodaj tablicę Renderable[] scene = new Renderable[10];
4. Wypełnij scenę różnymi obiektami i wykonaj render().

Zadanie 7
Bazując na hierarchii z Zadania 1 i Zadania 2:

- Stwórz metodę: public static Vehicle[] filterElectric(Vehicle[] vehicles)
    - która zwraca nową tablicę tylko z tymi pojazdami, które implementują Electric.
- Stwórz drugą metodę: public static Vehicle[] filterByYear(Vehicle[] vehicles, int year)
    - która zwraca nową tablicę pojazdami z danego roku
- Przetestuj filtrację na tablicy pojazdów o wymieszanych typach.

Laboratorium 7

Temat: Przetwarzanie kolekcji obiektów

Zadanie 1
Utwórz klasę:
- class Product { String name; double price; String category; boolean discounted; }

Utwórz kolekcję:
- ArrayList<Product> inventory

Następnie:
1. dodaj produkty spożywcze do kolekcji (min. 10) z losową ceną z przedziału <5, 100>
2. wypisz wszystkie produkty,
3. przefiltruj produkty z kategorii np. „nabiał” tańsze niż 5 zł,
4. ustaw discounted = true dla wszystkich produktów droższych niż 40 zł,
5. usuń wszystkie produkty o nazwie np. „Chleb” (jeśli istnieją),
6. posortuj produkty:
    a. rosnąco po cenie,
    b. malejąco po cenie,
    c. alfabetycznie po nazwie.

Zadanie 2
Utwórz klasa:
- class Book { String title; String author; int year; boolean borrowed; }

Utwórz kolekcje:
- ArrayList<Book> books
- HashMap<String, ArrayList<Book>> booksByAuthor

Dla zdefiniowanych kolekcji wykonaj:
1. dodaj kilka książek do listy books,
2. wykorzystując kolekcję books zbuduj mapę autor → lista jego książek,
3. wypisz wszystkich autorów oraz liczbę ich książek,
4. znajdź autora o największej liczbie pozycji,
5. wypisz książki starsze niż X lat,
6. wypisz wypożyczone książki
7. posegreguj całą listę książek rosnąco po roku wydania.

Zadanie 3
Zaimplementuj klasę:

- class Customer { String name; int serviceTime; int waitingTime}

Następnie utwórz kolekcję:

- Queue<Customer> queue = new LinkedList<>();

Następnie :

1. dodaj kilku klientów do kolejki,
2. zasymuluj obsługę:
    a. dopóki kolejka nie jest pusta:
        i. pobierz klienta z kolejki,
        ii. wypisz: „Obsługuję klienta X (czas: Y min)”.
3. zmodyfikuj symulację tak, aby po każdej obsłudze czas oczekiwania reszty klientów w kolejce zwiększał się o czas obsługi.
4. gdy kolejka jest pusta, wyświetl średni czas obsługi i średni czas oczekiwania

Zadanie 4
Zdefiniuj klasę:

- class Ticket { int id; String title; String user; int priority; boolean resolved; }

Utwórz kolekcje:

- ArrayList<Ticket> tickets
- Queue<Ticket> highPriorityQueue — obsługa priorytetów 9–10
- Queue<Ticket> normalQueue — reszta zgłoszeń

Następnie wykonaj:

- utwórz losowe zgłoszenia o różnych priorytetach,
- dodaj je do odpowiednich kolejek,
- napisz metodę handleNextTicket(), która:
    - najpierw obsługuje highPriorityQueue,
    - jeśli pusta — obsługuje normalQueue,
- usuń ze wszystkich kolekcji zgłoszenia rozwiązane,
- znajdź w całej kolekcji zgłoszenie użytkownika X.

Zadanie 5

Zdefiniuj klasy:

- abstract class Passenger { String name; boolean checkedIn; }
- class RegularPassenger extends Passenger { boolean hasExtraLuggage; }
- class VipPassenger extends Passenger { String loungeAccessCode; }

Zdefiniuj kolekcję:

- ArrayList<Passenger> passengers

Następnie:

1. dodaj losowych pasażerów,
2. zaznacz „checked in” dla wszystkich VIP
3. zaznacz “checked in” dla wybranych zwykłych pasażerów
4. usuń pasażerów, którzy nie dokonali odprawy,
5. przy pomocy instanceof wypisz:
    a. ilu VIPów jest wśród pasażerów,
    b. ilu zwykłych pasażerów ma dodatkowy bagażu
6. stwórz osobną kolejkę Queue<Passenger> reprezentującą kolejkę do bramki. Przenieś tu pasażerów, którzy są po odprawie z listy passengers. VIP zawsze wchodzi na początek kolejki.

Zadanie 6

Zdefiniuj klasy:

- class Item { UUID itemId; String name; int quantity; }
- class Order { UUID orderId; UUID itemId, int amount; }

Utwórz kolekcje:

- ArrayList<Item> inventory
- Queue<Order> orders

Następnie:

1. dodaj kilka pozycji (Item) do magazynu (inventory),
2. wygeneruj losowe zamówienia do kolejki,
3. następnie przeprowadź symulację realizacji zamówienia:
    a. pobierz kolejne zamówienie
    b. sprawdź, czy jest wystarczająca ilość w magazynie
        i. jeśli tak → zmniejsz stan magazynowy
        ii. jeśli nie → wypisz komunikat i zwróć zamówienie na koniec kolejki
    c. zakończ symulację, gdy:
        i. kolejka nie zmienia się między dwoma iteracjami lub wszystkie zamówienia zostały obsłużone.

Zadanie 7
Zdefiniuj klasę:

- class Transaction { String user; double amount; String currency; long timestamp; }

Utwórz następujące kolekcje:

- ArrayList<Transaction> history
- HashMap<String, ArrayList<Transaction>> groupedByUser

Następnie:

1. dodaj transakcje różnych użytkowników,
2. zbuduj mapę user -> lista transakcji,
3. znajdź użytkownika o największej liczbie transakcji,
4. oblicz sumę obrotu każdego użytkownika,
5. znajdź największą transakcję np. w Ethereum,
6. posortuj transakcje rosnąco po timestamp.

Zadanie 8
Zdefiniuj klasy:

- class User { String name; ArrayList<String> watchedGenres; }
- class Movie { String title; String genre; double rating; }

Następnie zdefiniuj kolekcje:

- ArrayList<User> users
- ArrayList<Movie> movies
- HashMap<String, ArrayList<Movie>> moviesByGenre

Następnie:

1. wygeneruj użytkowników i filmy i dodaj je do kolekcji,
2. zbuduj mapę genre → filmy (moviesByGenre),
3. dla każdego użytkownika wygeneruj listę rekomendacji:
    a. wybierz filmy z gatunków, które oglądał
4. dodaj symulację „do obejrzenia”:
    a. Queue<Movie> → filmy dodane przez użytkownika do obejrzenia.

Zadanie 9

Zdefiniuj kolekcję:

- ArrayList<Integer> numbers = new ArrayList<>();

Następnie:

1. Do listy dodaj kilkanaście losowych liczb całkowitych (np. z zakresu 1–100).
2. Zrealizuj poniższe zadania z wykorzystaniem strumieni https://www.geeksforgeeks.org/java/stream-in-java/
    - 2.1. Wypisz wszystkie liczby z kolekcji
    - 2.2. Wypisz tylko liczby parzyste
    - 2.3. Policz liczbę liczb większych niż 50
    - 2.4. Oblicz średnią wszystkich liczb w kolekcji
    - 2.5. Znajdź największą liczbę
    - 2.6. Stwórz nową listę, która będzie się składać z liczb pomnożonych przez 2 z kolekcji numbers

Laboratorium 8

Temat: Strumienie wejściowe/wyjściowe / przetwarzanie plików / serializacja.

Zadanie 1

Napisz program, który:

1. Wczytuje ścieżkę do pliku tekstowego z:
   - 1.1. argumentu wiersza poleceń lub
   - 1.2. jeśli argumentu nie ma – używa domyślnej ścieżki data/input.txt.
2. Otwiera plik i wypisuje jego zawartość na ekran:
   - 2.1. z numerami linii (format: 1: jakaś treść linii),
   - 2.2. pomijając puste linie (same spacje też traktujemy jako puste).,
   - 2.3. Po zakończeniu wypisywania:
     - 2.3.1. wyświetla liczbę wczytanych (niepustych) linii.

```
“treść linii 1

treść linii 2”

wyświetli w formie:
“1:treść linii 1
2:treść linii 2
liczba linii: 2“
```

Zadanie 2

1. Przygotuj plik app.log

```
2025-11-01 10:15:32 INFO User logged in
2025-11-01 10:16:01 ERROR Cannot connect to DB
2025-11-01 10:16:10 WARN Slow response
2025-11-01 10:17:45 ERROR Timeout while reading
```

2. Napisz program, który:
   - 2.1. Wczytuje plik logów linia po linii.
   - 2.2. Wszystkie linie zawierające słowo ERROR zapisuje do nowego pliku logs/errors.log.
   - 2.3. Na koniec:
     - 2.3.1. wypisuje na konsolę liczbę błędów,
     - 2.3.2. wypisuje informację „Plik błędów zapisano do: ...”.

Zadanie 3

1. Przygotuj plik products.csv

```
id;name;category;price
1;Klawiatura mechaniczna;peryferie;399.99
2;Mysz bezprzewodowa;peryferie;129.90
3;Monitor 24";monitory;799.00
…
```

2. Napisz program, który:
    - 2.1. Definiuje klasę Product z polami: id, name, category, price.
    - 2.2. Wczytuje plik CSV:
        - 2.2.1. pomija nagłówek,
        - 2.2.2. tworzy listę obiektów Product.
    - 2.3. Oblicza:
    - 2.4. łączną cenę wszystkich produktów,
    - 2.5. średnią cenę w każdej kategorii.
    - 2.6. Zapisuje wyniki do pliku data/report.txt w formacie np.:

```
Liczba produktów: 15
Suma cen: 5430.79

Średnia cena wg kategorii:
peryferie -> 220.45
monitory -> 1020.30
```

Zadanie 4

Do serializacji należy wykorzystać zewnętrzną bibliotekę np. Gson lub Jackson.

1. Dodaj bibliotekę do projektu:
    a. przez Mavena/Gradle lub
    b. przez dołączenie zewnętrznego JAR do projektu w IDE.
2. Przygotuj krótkie README.txt (w katalogu projektu) z informacją:
    a. jaką bibliotekę wybrałeś,
    b. jak ją dodać do projektu (kroki w IDE / wpis w pom.xml).
3. Zdefiniuj klasę Task z polami title, description, isDone
4. Utwórz listę kilku zadań.
5. Zserializuj listę do JSON-a przy pomocy wybranej biblioteki do pliku data/tasks_lib.json.
6. Wczytaj ten plik z powrotem do List<Task> (deserializacja).
7. Wypisz wczytane zadania na konsolę.
8. Do klasy Task dodaj pole LocalDate dueDate i sprawdź, jak wybrana biblioteka sobie z tym radzi (czy trzeba dodatkowej konfiguracji).

Zadanie 5

Opracuj aplikację konsolową „ToDo Manager”, która:

1. Przy starcie:
    1.1. próbuje wczytać plik data/tasks.json (JSON – format zgodny z zadaniem 5) do kolekcji w repozytorium,
    1.2. jeśli plik nie istnieje, startuje z pustą listą zadań.
2. Umożliwia użytkownikowi w pętli:
    2.1. wyświetlenie listy zadań (z numerami wg kolejności),
    2.2. dodanie nowego zadania,
    2.3. oznaczenie zadania jako wykonane (done = true),
    2.4. zapis aktualnej listy zadań do pliku JSON,,
    2.5. wyjście z programu.

Przykładowe menu:

```
1. Pokaż zadania
2. Dodaj zadanie
3. Oznacz zadanie jako wykonane
4. Zapisz i wyjdź
Twój wybór: _
```

3. Przy opcji „Zapisz i wyjdź”:
    3.1. serializuje listę zadań z repozytorium do JSON-a (zewnętrzna biblioteka),
    3.2. zamyka program.

Wymagania architektoniczne

1. wydziel przynajmniej dwie warstwy:
    1.1. logika biznesowa (TaskService – operacje na liście zadań),
    1.2. warstwa przechowywania danych (TaskRepository) - ma zawierać tylko kolekcję zadań,
2. mała warstwa prezentacji (np. ConsoleUI – drukowanie menu, pobieranie wyboru użytkownika).

Zadanie 6 (dla chętnych)

1. Rozszerz aplikację z zadania 5:
2. Przy każdym zapisie tasks.json zrób kopię:
    2.1. jeśli istnieje poprzednia wersja pliku, przenieś ją do backup/tasks-yyyy-MM-dd-HH-mm-ss.json,
    2.2. dopiero potem zapisz nową wersję.
3. Pozwól użytkownikowi z menu:
    3.1. wyświetlić listę dostępnych backupów,
    3.2. wybrać backup, z którego dane zostaną przywrócone jako aktualna lista zadań.

Laboratorium 9

Temat: Obsługa wyjątków

Zadanie 1
Napisz program, który:
- pobiera z konsoli dwie liczby całkowite,
- dzieli pierwszą liczbę przez drugą,
- obsługuje błąd dzielenia przez zero.

Zadanie 2
Rozszerz program z zadania 1:
- obsłuż osobno:
  - NumberFormatException,
  - ArithmeticException.

Wymagania:
- użyj co najmniej dwóch bloków catch,
- dla każdego wyjątku wypisz inny komunikat.

Zadanie 3
Napisz metodę:
- static int calculateAge(int birthYear)

Zasady:
- jeżeli birthYear > 2025, rzuć IllegalArgumentException,
- w przeciwnym razie zwróć obliczony wiek.

Wymagania:
- użyj instrukcji throw,
- obsłuż wyjątek w metodzie main.

Zadanie 4
Utwórz własny wyjątek:
- class UnderageException extends Exception

Napisz metodę:
- static void checkAge(int age) throws UnderageException

Zasady:
- jeżeli age < 18, rzuć TooYoungException,
- obsłuż wyjątek w metodzie main.

Zadanie 5
Napisz dwie metody:
- static void methodA() throws UnderageException
- static void methodB() throws UnderageException

Wymagania:
- methodA() wywołuje methodB(),
- methodB() rzuca UnderageException ,
- wyjątek jest obsłużony dopiero w main.